Accelerating SpGEMM Through Lightweight Clustering and Cache-Aware Accumulation

Shengle Lin*†, Chubo Liu*†, Yan Ding*†, Yikun Hu*†, Wangdong Yang*†, Keqin Li‡, and Kenli Li*†
*College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China
†National Supercomputing Center in Changsha, Changsha 410082, China
‡Department of Computer Science, State University of New York, New Paltz, New York 12561, USA
Email:{lsl036, liuchubo, ding, yikunhu, yangwangdong}@hnu.edu.cn, lik@newpaltz.edu, lkl@hnu.edu.cn

***Abstract*—Sparse matrix-matrix multiplication is one of the most crucial kernels in linear algebra solvers, graph analytics, and machine learning workloads. The clustered row-row style has emerged as a favorable SpGEMM workflow because of its potentially good parallelism. However, due to frequent data movement caused by irregular memory access, existing approaches often suffer from three issues: (1) workload imbalance and prohibitive preprocessing overhead, (2) unsatisfactory data locality, and (3) inefficient accumulation strategy.**

To this end, we propose LeSpGEMM, a Locality-enhanced SpGEMM algorithm. LeSpGEMM introduces a lightweight row-clustering method to improve spatial locality with near-linear complexity, and a cache-aware hybrid accumulation strategy to exploit data reuse across diverse sparse patterns. It further incorporates optimized kernels, including a SIMD-accelerated dense accumulator and an efficient hash-based accumulator. Experimental results on multiple platforms show that LeSpGEMM significantly outperforms six state-of-the-art implementations while maintaining low preprocessing overhead.

***Index Terms*—SpGEMM, sparse matrices, parallel algorithms, linear algebra, clustering methods, multicore processing**

I. Introduction

Sparse matrix-matrix multiplication (SpGEMM) computes the product of two sparse matrices A and B, producing a sparse matrix C. As a fundamental kernel in sparse linear algebra, SpGEMM critically impacts the performance of a wide range of applications, including algebraic multigrid solvers [1]–[3], graph analytics [4]–[7], and sparse deep neural networks [8]–[11]. However, due to the sparsity of both input and output matrices, SpGEMM suffers from highly irregular memory accesses and excessive data movement [12]–[14]. These bottlenecks lead to significant performance challenges on modern multicore architectures designed for regular memory access patterns and high spatial locality. As a result, data reuse targeting the L2 cache has become a key optimization direction in state-of-the-art SpGEMM approaches [15]–[21].

Currently, most parallel SpGEMM algorithms adopt the Gustavson row-row workflow [22], which computes each row of C by merging rows of B indexed by the nonzero columns of the corresponding row in A. To improve memory locality, prior work reorders the rows of A to enhance data reuse of B, leveraging techniques such as graph partitioning (GP) [23]–[25], hypergraph partitioning (HP) [26], spectral analysis [18], and approximate similarity [15], [19], [27]. However, **these approaches face the dilemma of high preprocessing overhead or low reordering quality.** We conduct a preliminary study of this limitation, as shown in Fig. 1. Although GP and HP are widely regarded as the most effective reordering strategies [17], [19], GP exhibits unstable performance, ranging from $0.82\times$ to $1.26\times$, while HP incurs orders-of-magnitude higher preprocessing overhead, rendering both approaches suboptimal in practice. Hierarchical clustering [19] offers stable speedups but suffers from execution failures on datasets like *wikipedia-20070206* due to prohibitive memory overheads from intermediate products.

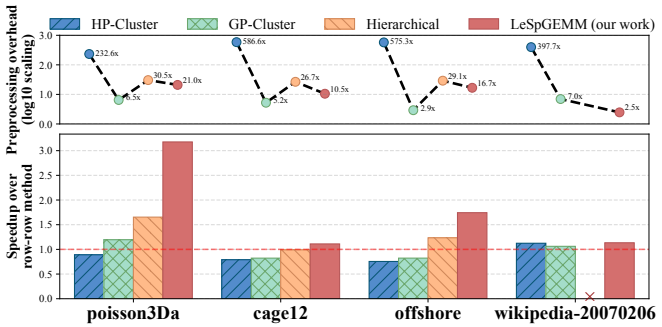


Fig. 1: A brief observation on the performance improvement and preprocessing overhead of existing SpGEMM algorithms.

Another critical factor is the diversity of non-zero patterns in sparse matrices, which induces a wide range of data flows in SpGEMM. A variety of sparse accumulator designs, including heap-based [29], [30], hash-based [31]–[36], dense [12], [37]–[39], and sort-merge [21], [40], [41] algorithms, exhibit performance degradation on certain input rows. Therefore, **an efficient accumulator design must adapt to both data locality and input variability**.

To this end, we present LeSpGEMM, a Locality-enhanced SpGEMM algorithm that bridges the gap between irregular memory access patterns and hierarchical memory systems on CPUs. LeSpGEMM is built upon two tightly integrated optimizations. ❶ Lightweight two-phase clustering. We propose a locality-aware clustering scheme based on *Constrained Locality Sensitive Hashing (C-LSH)*. It first leverages C-LSH to efficiently identify candidate row pairs, which are then aggregated to construct high-quality row clusters. ❷ Cache-

SC26, November 15-20, 2026, Chicago, Illinois, USA
979-8-3195-4789-7/26/$31.00 ©2026 IEEE

TABLE I: Time and space complexity analysis of row reordering or clustering algorithms for SpGEMM.

Terms	Method	Phase i: Candidate Gen.	Phase ii: Clustering	Memory Space	Scalability
M, N - number of rows, cols $nnz(A)$ - number of nonzeros I, l, p - number of iterations, layers, and partitions b - number of bands ω - constrained window size K - length of hash signature $\mathcal{P}$ - all candidate pairs dr_i - degree of row i	GP (METIS) [24]	N/A	$O(M + nnz(A))$	$O(M + nnz(A))$	High
	HP (PaToH) [28]	N/A	$O(I \cdot nnz(A) \cdot l \cdot p)$	$O(nnz(A) + M + N)$	Impractical
	Hierarchical [19]	$O(\sum dr_i^2)$ (by AA^T)	$O(\lvert\mathcal{P}\rvert \log \lvert\mathcal{P}\rvert)$	$O(nnz(AA^T) + \lvert\mathcal{P}\rvert)$	Vulnerable
	LSH [27]	$O(K \cdot nnz(A) + \lvert\mathcal{P}\rvert)$	$O(\lvert\mathcal{P}\rvert \log \lvert\mathcal{P}\rvert)$	$O(M \cdot K + \lvert\mathcal{P}\rvert)$	Good
	C-LSH (Ours)	$\mathbf{O(K \cdot nnz(A) + Mb\omega)}$	$\mathbf{O(M \cdot b\omega \log(M \cdot b\omega))}$	$\mathbf{O(M \cdot (K + b\omega))}$	**Excellent**

aware hybrid accumulation. We design an architecture-centric execution layer with a cache-aware adaptive dispatcher, which dynamically routes workloads between a robust hash-based accumulator and a vectorized *Block-SPA* (sparse accumulator). This design improves SIMD utilization and maximizes *L2 cache* residency through adaptive kernel selection.

By testing 632 square matrices from the Suitesparse Collection [42] with a number of nonzeros (nnz) ranging from 100K to 2M, no less than one billion floating-point operations for calculating $C = A^2$, our LeSpGEMM outperforms six state-of-the-art algorithms: HaSpGEMM [32], Intel MKL [43] (AMD AOCL [44]), Kokkos-Kernels [45], [46], CSeg [47], Hierarchical-clustering SpGEMM [19], and SuiteSparse:GraphBLAS [5], [7]. The experimental results demonstrate that LeSpGEMM is faster than these methods on 621, 575, 614, 480, 492, and 507 matrices, achieving average speedups of $8.58\times$, $1.97\times$, $2.48\times$, $1.20\times$, $1.56\times$, and $2.07\times$, respectively. Furthermore, ablation studies also reveal that the proposed lightweight clustering achieves a favorable balance between preprocessing overhead and clustering quality. The main contributions of this work are as follows:

- We address three major performance challenges in Gustavson's row-row style SpGEMM.
- We propose LeSpGEMM, a CPU-oriented SpGEMM algorithm that integrates lightweight clustering with a cache-aware hybrid accumulation strategy.
- We demonstrate consistent performance improvements over six state-of-the-art implementations across a diverse set of sparse matrices on three platforms.

II. Background and Motivation

A. SpGEMM and Its Clustering Algorithm

Sparse matrix-matrix multiplication (SpGEMM) computes $C = AB$, where A, B, and C are sparse matrices. Fig. 2 shows an example of multiplying a 6×6 matrix A with a 6×4 matrix B. In the Gustavson row-wise formulation, each row of C is computed by accumulating scaled rows of B indexed by the nonzero column indices of the corresponding row in A. As shown in Fig. 2, row 0 of A contains nonzeros in columns 0 and 5, thus the corresponding rows of B are accessed and accumulated. Since computations across rows of A are independent, this algorithm can be efficiently parallelized on multi-core architectures.

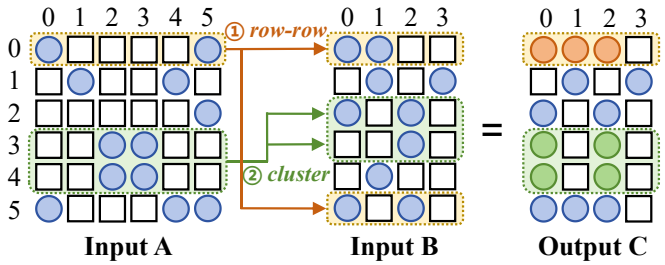


Fig. 2: An example of SpGEMM: ① the row-row workflow of Gustavson's algorithm; and ② its cluster-wise variation.

Recent studies show that retaining B in cache— particularly in the *L2 cache*—can significantly improve SpGEMM performance [15]–[21]. Accordingly, grouping rows of matrix A with similar patterns improves the data reuse of matrix B, thereby reducing redundant memory traffic and mitigating cache thrashing. As illustrated in Fig. 2, rows 3 and 4 of A are clustered because they share the same nonzero column indices 2 and 3. During computation, rows 2 and 3 of B can be reused from the L2 cache while accumulating the corresponding rows of matrix C. Cluster-based SpGEMM can be broadly categorized into fixed-length and variable-length schemes. Due to greater flexibility and storage efficiency, variable-length schemes are widely adopted in state-of-the-art SpGEMM algorithms [15], [18], [19].

In practice, clustering does not strictly require identical nonzero patterns across rows; instead, it commonly relies on *Jaccard Similarity* [48] as a metric:

$$J(S_i, S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}, \tag{1}$$

where S_i denotes the set of column indices corresponding to nonzeros in row i of A. For example, in Fig. 2, row 0 has nonzeros in columns 0 and 5 (i.e. $S_0 = \{0, 5\}$), while row 2 has a nonzero only in column 5 (i.e. $S_2 = \{5\}$). Their Jaccard similarity is $|S_0 \cap S_2|/|S_0 \cup S_2| = 0.5$. While Jaccard similarity provides a theoretical foundation for row clustering, efficiently identifying similar rows without incurring high preprocessing overhead remains a key challenge.

B. Row Reordering and Clustering

A straightforward approach to computing and storing pairwise similarity among M rows requires $O(M^2)$ time and

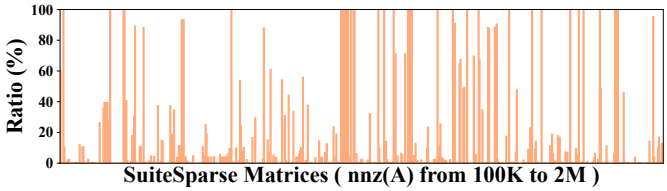


Fig. 3: Ratio of compact clusters ($compactness > 50\%$) in resulting matrix C when computing $C = AA$.

memory, which is impractical for real-world applications [49]. To address this limitation, various reordering and clustering heuristics have been proposed [50]. These approaches aim to reduce the search space for similar rows while keeping preprocessing overhead manageable.

Graph partitioning methods. Among existing sparse reordering techniques, graph partitioning (GP) has been verified as effective for SpGEMM [19]. As shown in Table I and illustrated in Fig. 1, GP (METIS) [23], [24] achieves near-linear complexity but often fails to deliver consistent performance improvements. In contrast, HP (PaToH) [26], [28] captures row similarity more accurately. However, its iterative refinement process and data dependency introduce substantial computational overhead, limiting parallel scalability. More importantly, these methods are primarily designed for load balancing rather than data locality optimization, which restricts their effectiveness across diverse sparse patterns.

Intersection-based similarity computation. An alternative paradigm identifies similar rows by explicitly computing structural intersections. This method typically performs a symbolic AA^T to evaluate pairwise overlaps between rows [18], [19]. Compared with graph partitioning, it captures data reuse opportunities more precisely. However, it reformulates the reordering problem as a preprocessing SpGEMM task. As shown in Table I, this process incurs a complexity of $O(\sum dr_i^2)$, which becomes prohibitive for matrices with skewed degree distributions. In particular, for power-law matrices [21], [51], high-degree rows dominate the computation, leading to a rapid growth of intermediate results. Since all intersections must be explicitly enumerated, this often results in excessive memory consumption and unpredictable preprocessing latency.

Global searching strategies. To alleviate the high dimensionality of similarity search, locality-sensitive hashing (LSH) [52] is widely adopted to approximate row similarity. However, it typically relies on global search for exhaustive candidate generation [27], which is highly sensitive to the sparse pattern of A. For matrices that already exhibit clustered structures, the number of candidate pairs $\mathcal{P}$ can explode combinatorially, reaching $O(M^2)$ in the worst case. This lack of an explicit bound on candidate size leads to unpredictable time and space costs, making it difficult to achieve a stable trade-off between preprocessing overhead and runtime performance.

C. Compactness of Clusters

Variations in sparse patterns introduce significant algorithmic diversity in SpGEMM, even among rows within a single matrix [53]–[55]. To quantify locality potential, we define the compactness of a row cluster as the ratio of *nnz* to its bounding rectangular area:

$$Compactness = \frac{nnz(cluster)}{cluster_height \cdot col_span}, \tag{2}$$

where $cluster_height$ denotes the number of rows in the cluster, and col_span is the number of nonzero columns.

To assess data reuse in practice, we analyze the compactness distribution of clusters in $C = A^2$ over the matrices from the SuiteSparse Collection. As illustrated in Fig. 3, nearly 60% of datasets contain a substantial fraction of clusters with compactness exceeding 50%. Such compact clusters indicate that partial products are concentrated within narrow column ranges, resulting in frequent address reuse and strong spatial locality during accumulation.

However, conventional sparse accumulators fail to fully exploit this locality, as they typically treat each partial product as an independent sparse update [17], [19], [32]. This mismatch between data compactness and computing behavior necessitates a specialized accumulator design for efficiently processing such compact dataflows in SpGEMM.

D. Motivation of This Work

Based on the aforementioned analysis, we identify two key challenges in current cluster-wise SpGEMM:

1) **Trade-off between clustering quality and preprocessing cost.** The performance of cluster-wise SpGEMM critically depends on clustering quality, which determines memory footprint and data reuse. However, existing approaches suffer from either limited scalability or high computational complexity. Large constant factors, excessive intermediate results, and an unpredictable number of candidate pairs pose significant challenges for this problem. There is no existing reordering framework that achieves high-quality clustering with deterministic, near-linear complexity for large-scale matrices.

2) **Underutilized locality in compact clusters.** Our observation reveals that post-clustering matrices often exhibit a high fraction of compact clusters. Nevertheless, conventional accumulation schemes—such as heap, hash, or SPA—are designed for irregular sparse updates and treat intermediate products as independent entries. This design fails to exploit spatial locality and ignores vectorization opportunities on modern processors, resulting in suboptimal hardware utilization even when strong structural locality exists.

To address these challenges, a co-designed solution is required. First, a lightweight clustering method is needed to balance quality and overhead with deterministic complexity. Second, the accumulation process must be reconsidered to fully leverage compact data layouts, enabling efficient cache residency and vectorized acceleration. In the next section, we present the LeSpGEMM algorithm.

III. LeSpGEMM

A. Overview

Our LeSpGEMM is a locality-enhanced SpGEMM algorithm that integrates lightweight clustering with cache-aware

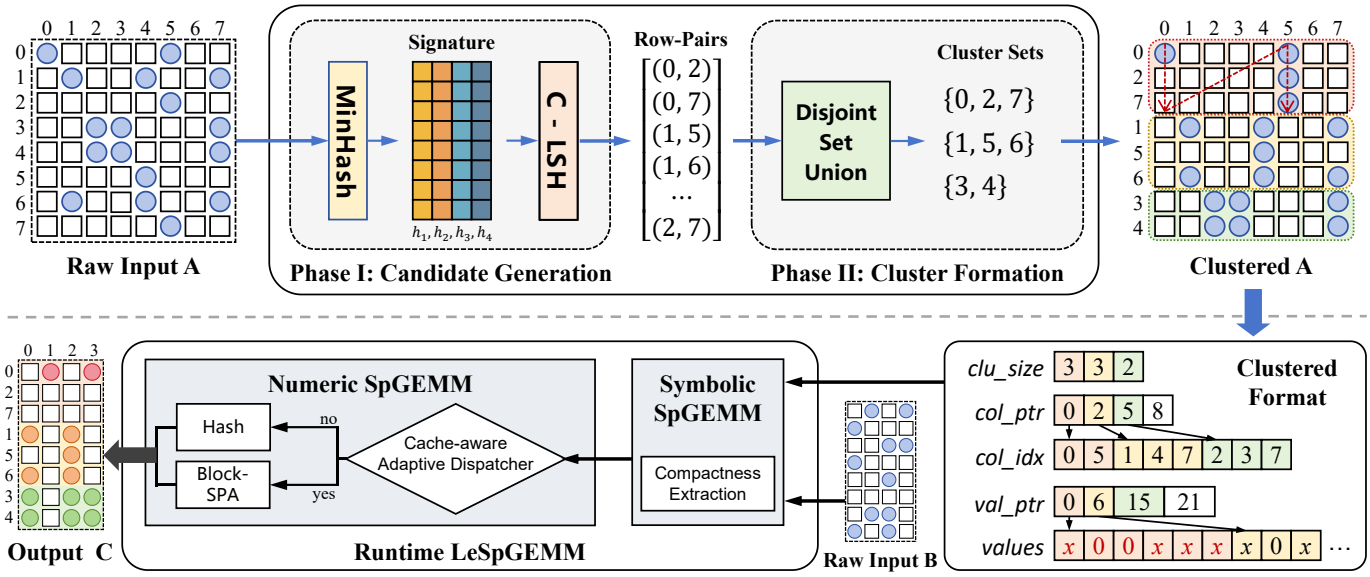


Fig. 4: An overview of LeSpGEMM, where the upper half performs row-clustering preprocessing on the sparse input A, and the lower half shows the runtime execution of LeSpGEMM computing $C = AB$.

hybrid accumulation. Its overall architecture is illustrated in Fig. 4, which consists of two primary stages: preprocessing and SpGEMM execution.

The upper part of Fig. 4 shows the preprocessing stage on matrix A, including deterministic candidate pairs generation via C-LSH and clustering using disjoint-set union. This stage enables the sparse matrix A to exhibit **coarse-grained locality**, meaning that rows of A within the same cluster share similar access patterns to matrix B. We further adopt a column-major storage format within each cluster, which facilitates the reuse of rows of B during computation.

The lower part of Fig. 4 presents the runtime LeSpGEMM execution. After symbolic analysis, LeSpGEMM employs a cache-aware adaptive dispatcher to exploit **fine-grained locality**. Specifically, it selects the most suitable accumulator based on cache budget and cluster compactness. Overall, this two-stage optimization bridges sparse structures with cache-efficient execution, providing robust and significant acceleration across diverse sparse matrices.

B. Lightweight Two-phase Clustering

*1) **Phase I: Constrained-LSH via Row Signatures**:* The first phase identifies potential row similarities with low computational overhead. Each sparse row R_i is compressed into a fixed-length signature vector Sig_i using *MinHash* [56]. By applying K independent hash functions, MinHash maps high-dimensional column indices into a compact signature space, as shown in Fig. 4 where $K = 4$. The time complexity of this process is $O(K \cdot nnz(A))$. Theoretically, this mapping provides an unbiased estimator of Jaccard similarity, where the signature serves as a robust proxy for structural affinity [52].

Following standard locality-sensitive hashing (LSH), each signature is partitioned into b bands with r signatures ($K = b \times r$). Rows sharing an identical sub-signature in any band are hashed into the same bucket $\mathcal{B}$. However, for highly skewed power-law matrices, naive global pairing within large buckets can cause bucket explosion, leading to up to $O(M^2)$

Algorithm 1: Candidate Pairs Generation by C-LSH

Input : Sparse matrix $\mathbf{A}$; Signature length K; Bands b; Bucket limit bk_l; Window size ω.
Output: Candidate Set $\mathcal{P} = \{(i, j, sim_{ij})\}$.

Initialize $\mathbf{Sig} \leftarrow \{\infty\}^{M \times K}$, $\mathcal{P} \leftarrow \emptyset$;
// MinHash Signature
for *each* $a_{i,j}$ *in* $\mathbf{A}$ ***in parallel*** **do**
  for $k \leftarrow 0$ **to** $K - 1$ **do**
    $\mathbf{Sig}_i[k] \leftarrow \min(\mathbf{Sig}_i[k], hash_k(j))$;
// Constrained Row-pair Generation
for *each band* $\beta \in \{0, \ldots, b-1\}$ ***in parallel*** **do**
  Construct hash table $\mathcal{H}_\beta$ based on signature bands;
  for *each bucket* $\mathcal{B} \in \mathcal{H}_\beta$ **do**
    if $|\mathcal{B}| \leq bk_l$ **then**
      // Capture All Pairs
      $sim_{ij} \leftarrow est_similarity(\mathbf{Sig}_i, \mathbf{Sig}_j)$;
      $\mathcal{P} \leftarrow \mathcal{P} \cup \{(i, j, sim_{ij}) \mid i, j \in \mathcal{B}, i < j\}$;
    else
      // Sliding-window Strategy
      for $l \leftarrow 0$ **to** $|\mathcal{B}| - 1$ **do**
        for $k \leftarrow 1$ **to** $\min(\omega, |\mathcal{B}| - l - 1)$ **do**
          $i \leftarrow \mathcal{B}[l], j \leftarrow \mathcal{B}[l + k]$;
          $\mathcal{P} \leftarrow \mathcal{P} \cup \{(i, j, sim_{ij})\}$;
return $\mathcal{P}$;

complexity. This violates the requirement for deterministic preprocessing in large-scale applications.

To resolve this issue, we propose Constrained-LSH (C-LSH). As outlined in Algorithm 1, C-LSH generates candidate row pairs under explicit capacity constraints while recording the estimated hash similarity sim for each pair:

- *Bucket-size thresholding*: For buckets whose size is below a predefined threshold, we perform full Cartesian product comparisons to preserve recall (256 in our experiments; bk_l in Algo. 1, lines 8-9).
- *Window-constrained search*: For those oversized buckets, we adopt a sliding-window strategy where each row evaluates its estimated similarity only with a limited number of subsequent neighbors (16 in our experiments; ω in Algo. 1, lines 13-14).

Consequently, C-LSH decouples the preprocessing overhead from the sparsity distribution of the input matrix, effectively bounding the total candidate generation complexity by $O(M \cdot b\omega)$. Therefore, this mechanism eliminates the quadratic candidate explosion observed in naive LSH for skewed datasets, ensuring deterministic and memory-safe preprocessing.

*2) **Phase II: Union-Find Based Clustering**:* In the second phase, the candidate pairs are aggregated into disjoint clusters to determine the final row permutation and clustering of A. To achieve high efficiency, we employ a *disjoint set union* (DSU) data structure, which is also known as *union-find*.

Algorithm 2 presents the reordering and clustering procedure. The process begins by initializing a max-priority queue $\mathcal{Q}$ with all candidate triplets $(i, j, sim_{ij}) \in \mathcal{P}$ generated in Phase I (Algo. 2, line 1). At each iteration, the pair with the highest similarity is extracted to drive a greedy merging process (Algo. 2, line 5). In DSU, each cluster is represented as a tree, and merging is performed by attaching one tree as a subtree of another. For efficiency, the structural pattern of each cluster is represented by its root node. Initially, each row represents an independent cluster of its own. To maintain structural consistency while reducing redundant computations, two merging scenarios are considered:

(1) *Direct representative merging.* If both row i and row j are representatives of their respective clusters, the smaller cluster is merged into the larger one via a *Union* operation (Algo. 2, line 10). To balance accuracy and parallelism, the cluster size is limited by a predefined threshold (8 in our experiments; C_{max} in Algo. 2, lines 11-12). Once the threshold is reached, the cluster is marked as saturated and excluded from further merging (Algo. 2, line 12). This fast-path merging strategy leverages precomputed signature similarities sim_{ij} to accelerate early cluster formation.

(2) *Representative-based similarity refinement.* If either row i or row j already belongs to an existing cluster, their representatives r_i and r_j are first identified via path compression (Algo. 2, line 6). Since the similarity between clusters may shift during merging, the precise Jaccard similarity between r_i and r_j is calculated as a metric. The resulting score is then reinserted into $\mathcal{Q}$ for re-evaluation, thereby ensuring that subsequent merges are guided by ground-truth similarity rather than estimates (Algo. 2, lines 14-15).

Algorithm 2: Row Reordering and Clustering

Input : Candidate Set $\mathcal{P}$; Cluster size limit C_{max}.
Output: Row permutation $\mathcal{R}$; Cluster offsets $\mathcal{O}$.

```
// Initialization
1  Q ← Initialize max-priority queue with P.sim;
2  Parent, size ← Construct disjoint sets of M rows;
3  Valid ← {True}^M;
// Greedy Cluster Aggregation
4  while Q ≠ ∅ do
       // Get pair with largest est_similarity
5      (i, j, sim) ← Q.pop();
       // Path compression
6      r_i, r_j ← Find(i, Parent), Find(j, Parent);
7      if r_i == r_j or ¬Valid[r_i] or ¬Valid[r_j] then
8          continue;
       // Fast Path: Direct Union of roots
9      if i == r_i and j == r_j then
10         r_new ← Union(r_i, r_j, Parent, Size);
11         if Size[r_new] ≥ C_max then
12             Valid[r_new] ← False;
13     else
           // Add new pair to the queue
14         J(r_i, r_j) ← Jaccard(A[r_i, :], A[r_j, :]);
15         Q.push(r_i, r_j, J(r_i, r_j)) ;
// Permutation and Offset Generation
16 Identify cluster set C = {c_1, ..., c_k} from Parent;
17 Initialize R ← [ ], O_{k+1} ← {0};
18 for m ← 1 to k do
19     R ← R ‖ row indices in c_m ;    // Concatenate
20     O_m ← O_{m−1} + |c_m| ;          // Prefix-sum
21 return R, O;
```

In summary, the total time complexity of the second clustering phase (excluding C-LSH) is $O(|\mathcal{P}| \log |\mathcal{P}| + |\mathcal{P}| \log M + M)$, where $|\mathcal{P}| = O(M \cdot b\omega)$. Specifically, priority queue construction and subsequent greedy extraction incur $O(|\mathcal{P}| \log |\mathcal{P}|)$ complexity and dominate the overall cost (Algo. 2, lines 1, 5 and 15). The cluster management via DSU contributes $O(|\mathcal{P}| \log M)$ in the worst case (Algo. 2, lines 6-12), while exhibiting near-constant amortized complexity in practice. Finally, the permutation mapping is generated in $O(M)$ time (Algo. 2, lines 18-20). By constraining candidate pair generation, C-LSH ensures that the clustering phase remains computationally tractable even for already structured or highly skewed sparse matrices.

C. Clustered Storage Format

To efficiently store the clustered sparse matrix A, a column-block clustered sparse format is adopted. As illustrated in Fig. 4, this format consists of five arrays: clu_size, col_ptr, col_idx, val_ptr, and $values$. Specifically, the array clu_size

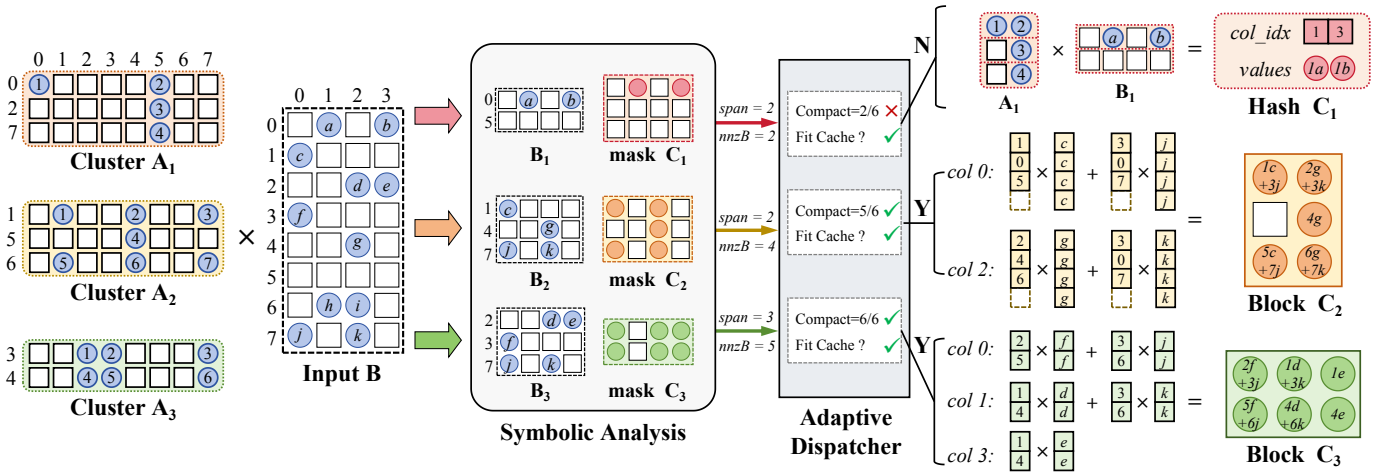


Fig. 5: An example of runtime LeSpGEMM execution. First, during the symbolic analysis, we can obtain the number of corresponding elements in B and the structure of C. Next, by analyzing cache utilization and compactness through a cache-aware adaptive dispatcher, LeSpGEMM calls different accumulators to complete the accumulations. In this example, we assume the L2 cache budget $L2_bg$ is $1MB$, and the compactness threshold τ is 0.5.

records the number of rows in each cluster, while col_ptr and val_ptr provide offsets for column indices and corresponding values, respectively. The key feature of this format lies in the column-major layout of the $values$ array. In contrast to the CSR format, this layout ensures that, when processing a column block, the corresponding rows of B remain resident in cache, thereby improving data locality.

To enable hardware-efficient execution, zero padding is introduced in the $values$ array to align each column block. In the provided example, the first cluster contains columns 0 and 5, resulting in an increment of 2 in col_ptr. The $values$ array is padded with two zeros to maintain a uniform block height, thereby enabling regularized loop unrolling and facilitating vectorized operations during the accumulation.

D. Cache-aware Hybrid Accumulation

As illustrated in Fig. 5, the execution of our LeSpGEMM begins with a symbolic analysis phase. During this stage, matrix A is processed at the cluster granularity. In addition to standard sparse pattern analysis and memory allocation for C, LeSpGEMM further extracts some critical structural metadata, including the required nnz of B and the column span of C. These lightweight metrics capture cache utilization potential and structural regularity, forming the decision basis for adaptive hybrid accumulation.

Based on these metrics, an adaptive dispatcher dynamically assigns each cluster to an appropriate execution path. Clusters with cache-friendly footprints and high compactness are processed via *Block-SPA*, enabling efficient register reuse and SIMD execution. In contrast, irregular or oversized clusters are handled by a hash-based accumulator to ensure robustness across diverse sparse patterns.

*1) **Adaptive Dispatcher Design**:* Motivated by the structural characteristics in Fig. 3, which reveal significant dense regions in the product C, we design a cache-aware adaptive dispatcher to improve accumulation efficiency.

For a cluster Ai, the dispatcher estimates the memory footprint $\mathcal{F}_i$ of Block-SPA execution. Specifically, this estimation models cache residency by accounting for data contention in the L2 cache. While the columns of A_i exhibit high temporal reuse, they are directly loaded into vector registers, thus exerting minimal pressure on the cache hierarchy. In contrast, the dense accumulator of C_i is the dominant component, serving as a stationary update target during accumulation. To prevent streamed rows of B from evicting the hot C_i block, our dispatcher evaluates the combined footprint of both computing components:

$$\mathcal{F}_i = Cost(B_i_rows) + Cost(C_i_acc), \tag{3}$$

where the $Cost(B_i_rows)$ represents the memory traffic from B_i, including the storage bytes required for both column indices and values. $Cost(C_i_acc)$ denotes the memory footprint required to maintain the dense output, computed as:

$$Cost(C_i_acc) = C_i_height \times C_i_span \times sizeof(entry). \tag{4}$$

The dispatcher enforces two constraints to select the execution path: ❶ *Compactness constraint:* The compactness must exceed a predefined threshold τ, ensuring that SIMD-aligned dense computation amortizes the overhead of zero padding. ❷ *Cache capacity constraint:* The estimated footprint $\mathcal{F}_i$ must fit within the L2 cache budget $L2_bg$, avoiding cache thrashing and excessive memory traffic.

Only clusters satisfying both constraints are dispatched to the Block-SPA path; otherwise, they are processed using the hash-based fallback to maintain robustness. Since all required features are pre-extracted during the symbolic phase, the adaptive dispatcher introduces only $O(M)$ overhead.

*2) **Block-SPA: Optimized SIMD Accumulation**:* If a cluster satisfies both dispatching criteria, LeSpGEMM invokes the Block-SPA kernel. As illustrated by C_2 and C_3 in Fig. 5, this kernel follows an outer-product formulation: a column of the clustered A is loaded into vector registers and multiplied with the corresponding scalar elements from B. By leveraging the aligned column-block layout, SIMD instructions (e.g., *AVX*) are utilized to process multiple rows simultaneously, significantly improving computational throughput. For clusters whose sizes are smaller than the vector width, zero padding is applied to enable full vector utilization, like C_2 in the example.

To maximize memory efficiency, intermediate results are accumulated into a stationary dense buffer that represents the bounding box of the output cluster C_i. The required memory footprint of C_i is given by Equation 4. In block-SPA, the fixed height of clusters enables aggressive loop unrolling and eliminates expensive dynamic index lookups and hash collisions. This design effectively bridges the gap between the irregular sparsity of input matrices and the regular access patterns required by modern processors.

*3) **Fallback Mechanism via Hash Accumulator**:* Clusters that do not satisfy the Block-SPA criteria—typically due to exceeding cache capacity or exhibiting low compactness—are processed using a hash-based accumulator. As illustrated by C_1 in Fig. 5, this mechanism serves as a robust fallback for irregular sparse patterns. In this path, a linear-probing hash table is employed to accumulate intermediate products generated from row-wise computations within a cluster. This approach is particularly effective when non-zero elements are distributed across a wide column range, as memory usage scales with the actual number of non-zero entries.

To ensure efficient parallel execution, LeSpGEMM incorporates a *load-balancing strategy* based on the number of intermediate products per cluster, assigning comparable workloads across threads. After accumulation, the results are written back to C using our clustered storage format, with the same row order as A. By integrating cache-aware hybrid accumulation with load balancing, LeSpGEMM achieves consistent performance across diverse sparse structures: the Block-SPA delivers high throughput for structured clusters, while the hash-based fallback path ensures memory efficiency and robustness for irregular or oversized clusters.

IV. Experiments

A. Experimental Setup

Platform. We evaluate LeSpGEMM on three multi-core architectures: ① Intel Xeon Gold 5120, a dual-socket system with 28 physical cores, 28 MiB of L2 cache, and 256 GB DDR4 memory; ② Intel Xeon Gold 6248, a single-socket system with 20 cores, 20 MiB of L2 cache, and 128 GB DDR4 memory; ③ AMD EPYC 7C13, a many-core processor with 64 physical cores, 32 MiB of L2 cache, and 512 GB DDR4 memory. All experiments are conducted using all available physical cores with hyper-threading disabled.

Dataset. We test 632 sparse square matrices from the Suitesparse Collection [42], with nnz ranging from 100K to 2M. In addition, we also evaluate 20 representative sparse matrices for an in-depth analysis. Table II summarizes their characteristics, where **CR** (compression rate) is defined as the ratio of **np**(C)—the number of intermediate products in $C = A^2$—to $nnz(C)$. Note that these matrices have been widely used in prior work [16], [19], [34] and are classic sparse datasets from diverse application domains.

TABLE II: Information of 20 representative sparse matrices.

Matrix A	Rows(A)	nnz(A)	np(C)	nnz(C)	CR
cant	62K	4.0M	269.5M	17.4M	15.5
pwtk	218K	11.6M	626.1M	32.8M	19.1
offshore	260K	4.2M	71.3M	23.4M	3.1
cage12	130K	2.0M	34.6M	15.2M	2.3
scircuit	171K	1.0M	8.7M	5.2M	1.7
wiki-Vote	8K	0.1M	8.7M	2.8M	3.1
poisson3Da	14K	0.4M	11.8M	3.0M	4.0
pdb1HYS	36K	4.3M	555.3M	19.6M	28.3
rma10	47K	2.4M	156.5M	7.9M	19.8
shipsec1	141K	7.8M	450.6M	24.1M	18.7
consph	83K	6.0M	463.8M	26.5M	17.5
filter3D	106K	2.7M	86.0M	20.2M	4.3
mac_econ_fwd500	207K	1.3M	15.6M	9.3M	1.7
af_shell10	1.5M	52.7M	1.8B	142.7M	12.9
hood	221K	10.8M	562.0M	34.2M	16.4
case39	40K	1.0M	4.0B	404.7M	10.0
gupta3	17K	9.3M	30.7B	270.9M	113.4
TSOPF_FS_b300_c2	57K	8.8M	53.9B	805.7M	67.0
com-LiveJournal	4M	69.4M	8.6B	4.9B	1.8
wikipedia-20070206	3.5M	45.0M	223.7B	165.7B	1.4

Implementation. LeSpGEMM is implemented in *C++17* with *OpenMP* for shared-memory parallelism. We use *AVX* instructions for SIMD vectorization and provide hardware-aware scripts to improve performance portability. We compare LeSpGEMM against six state-of-the-art SpGEMM implementations: HaSpGEMM [32], Intel MKL v2025.3.1 [43] (AOCL-Sparse v5.2 [44] on AMD CPUs), Kokkos-Kernels v5.0 [45], [46], CSeg [47], Hierarchical-clustering SpGEMM [19], and SuiteSparse:GraphBLAS v10.3.1 [5], [7], all in double precision. Each experiment is repeated 10 times, and the average performance is evaluated. All reported relative speedups are geometric means.

B. Microbenchmarks for Parameters

To determine the optimal values for $L2_bg$ and τ in the cache-aware adaptive dispatcher, we design two microbenchmarks for automatic parameter tuning.

First, we isolate the Block-SPA and hash accumulator from the entire LeSpGEMM pipeline to evaluate their performance under varying levels of compactness. This microbenchmark emulates a per-core L2 cache constrained environment and sweeps the block compactness from 0.02 to 0.50, as demonstrated in Fig. 7. Specifically, the generated data are carefully constructed to fit within the L2 cache during block accumulation, while the hash-based method performs an upward search in powers of two to allocate the hash table. From the intersection points of the two curves, we select $\tau = 0.16$, 0.28, and 0.30 for platforms ①, ②, and ③, respectively.

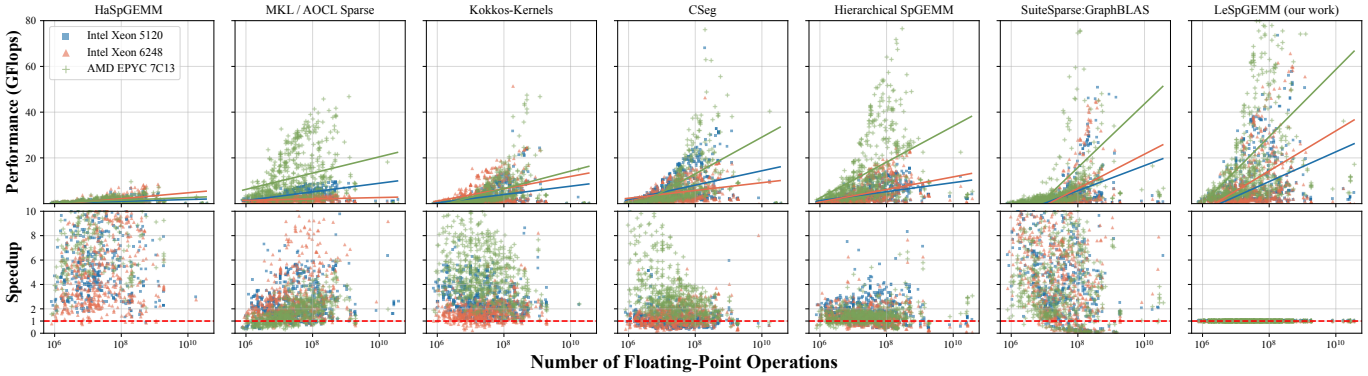


Fig. 6: Performance comparison of LeSpGEMM against six state-of-the-art SpGEMM algorithms on Intel Xeon 5120, Xeon 6248 and AMD EPYC 7C13 processors. The top row presents double precision performance of $C = A^2$ along with linear regression fits, while the bottom row shows the corresponding speedups of LeSpGEMM over competing methods.

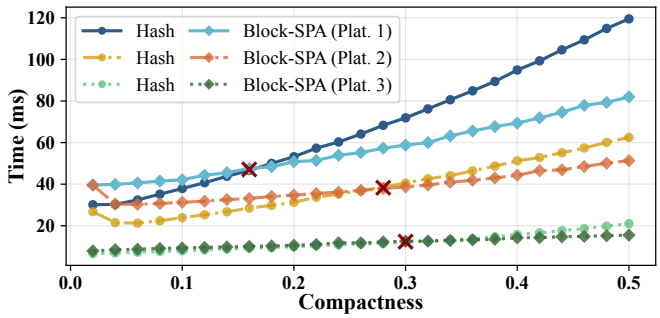


Fig. 7: Microbenchmark evaluation of two accumulators under varying compactness on three platforms.

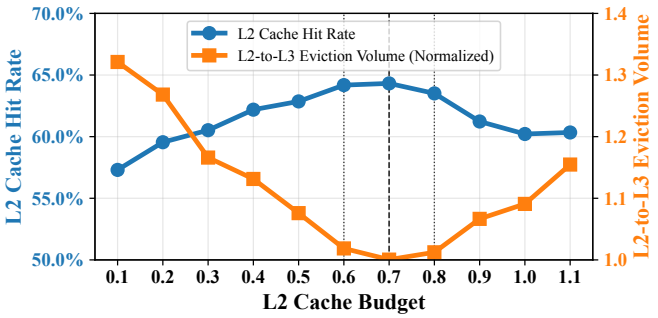


Fig. 8: L2 cache hit rate and normalized L2-to-L3 eviction volume versus the L2 cache budge on Intel Xeon 5120 CPU.

Next, we select matrices from Table II with a compact cluster ratio exceeding 90% to explore the L2 cache budget. During LeSpGEMM execution, we employ *perf-tool* [57] to measure the L2 cache hit rate and *likwid* [58], [59] to quantify the L2-to-L3 cache eviction volume. As illustrated in Fig. 8, a budget ratio in the range of $[0.6, 0.8]$ achieves both low eviction volume and high cache hit rate, with the optimal value observed at 0.7. Given that all platforms employ LRU-like replacement policies, we observe a consistent performance trend across architectures. Consequently, we set $L2_bg = 0.7$ for all subsequent experiments.

Notably, allocating the entire L2 capacity is counterproductive. If the accumulator fully occupies the cache, transient accesses from index streams and instruction fetches can inevitably evict its contents, leading to cache thrashing. By reserving a 30% margin, it ensures that the performance-critical blocks remain resident during the Block-SPA computation.

C. Performance Comparison over Existing Work

Fig. 6 reports the double-precision performance of $C = A^2$ across 632 datasets against HaSpGEMM, Intel MKL SpBLAS (or AOCL-Sparse), Kokkos-Kernels, CSeg, Hierarchical SpGEMM, SuiteSparse:GraphBLAS, and LeSpGEMM on three different platforms. Among these implementations, CSeg and Hierarchical SpGEMM are the most directly comparable baselines, as they also utilize segmentation or clustering to improve data locality. The remaining baselines include optimized SpGEMM implementations and widely used software libraries, representing practical state-of-the-art alternatives. For all methods, we evaluate steady-state SpGEMM time, excluding method-specific preprocessing costs, so the analysis focuses on computing-phase efficiency and robustness.

As shown in the top row of Fig. 6, only LeSpGEMM and hierarchical SpGEMM successfully process all 632 matrices. In contrast, HaSpGEMM, Kokkos, CSeg, MKL, and AOCL fail on 10 matrices due to memory exhaustion, while SuiteSparse:GraphBLAS fails on 5 matrices. Notably, our LeSpGEMM achieves a peak performance of 175.02 Gflops at *GT01R* on the AMD EPYC 7C13 CPU, outperforming the second-best result (AOCL-Sparse, 114.24 Gflops) by 53.20%. On the Xeon 5120, LeSpGEMM outperforms the six baselines on 621, 575, 614, 480, 492, and 507 matrices, respectively. Similar trends are observed on the other two platforms, where LeSpGEMM consistently delivers the best performance on the majority of the datasets.

Moreover, the bottom row of Fig. 6 presents the speedup distributions. On the Intel Xeon 5120, LeSpGEMM achieves geometric-mean speedups of 8.58×, 1.97×, 2.48×, 1.20×,

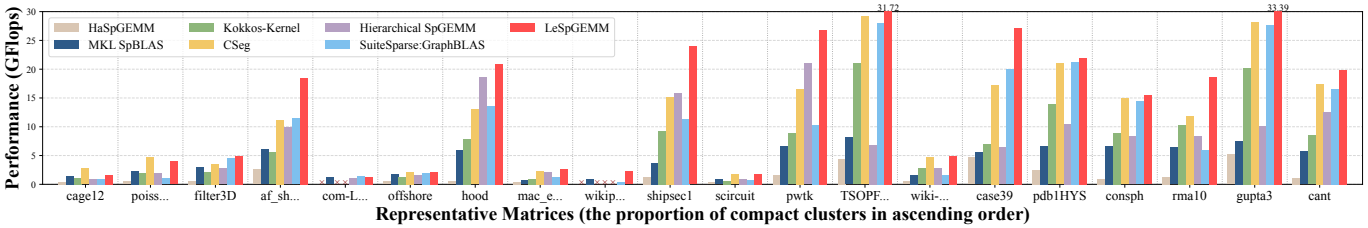


Fig. 9: A double-precision performance comparison on 20 representative matrices on Intel Xeon 5120 CPU.

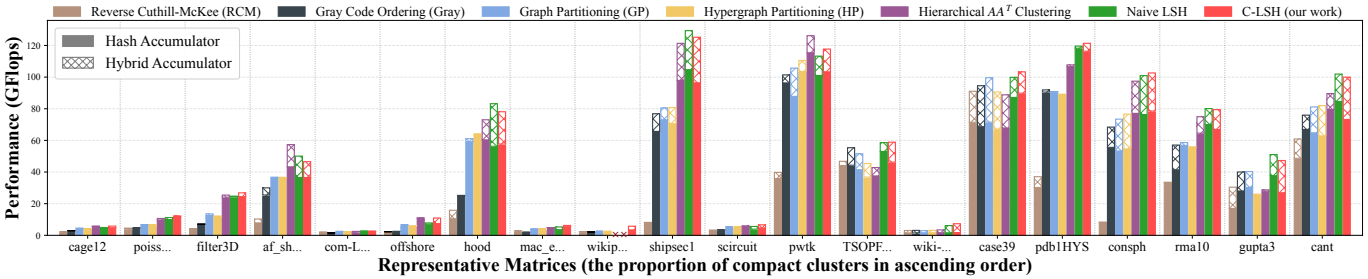


Fig. 10: The ablation study across several reordering algorithms with hash and hybrid accumulators on AMD EPYC 7C13.

1.56×, and 2.07× over the respective algorithms. On the Xeon 6248, the geometric-mean speedups remain significant at 5.98×, 2.73×, 1.72×, 1.30×, 1.30×, and 1.15×, respectively. Similarly, on the EPYC 7C13, the corresponding geometric-mean speedups are 13.94×, 1.47×, 4.41×, 2.58×, 1.34×, and 3.20×. Furthermore, as the number of floating-point operations increases, LeSpGEMM exhibits a favorable scaling trend, indicating higher efficiency for large-scale workloads.

For a finer-grained analysis, we present detailed results for 20 representative matrices (Table II) under the same steady-state timing scope on the Intel Xeon 5120. As shown in Fig 9, LeSpGEMM consistently outperforms six competitors in most instances when computing $C = A^2$. Notably, for matrices that cannot be processed by certain methods due to excessively large $nnz(C)$– such as *com-LiveJournal* and *wikipedia-20070206*– LeSpGEMM avoids the allocation of large intermediate products and maintains bounded auxiliary memory usage, thereby ensuring computational robustness. However, performance degradation is observed in several cases. Specifically, LeSpGEMM achieves 75% of the performance of SuiteSparse:GraphBLAS on *com-LiveJournal*, and 0.58× and 0.84× of CSeg on *cage12* and *poisson3Da*, respectively. These cases exhibit extremely low cluster compactness (approaching 0%), which prevents effective use of the hybrid accumulation strategy and forces a fallback to the hash-based accumulator. Overall, LeSpGEMM delivers a performance range of 0.58× to 2.36×, with a geometric mean of 1.12×, relative to the second-best implementation.

D. Ablation Study

To further evaluate the effectiveness of C-LSH and the hybrid accumulation strategy, we conduct an ablation study on the $C = AA^T$ workload. Fig. 10 first compares representative row-reordering and clustering schemes, including Reverse Cuthill-McKee (RCM) [60], Gray Code Ordering (Gray) [50], GP [24], HP [26], Hierarchical AA^T Clustering [19], naive LSH [27], and our C-LSH, under the same hash-based accumulator. This setting isolates the effect of each reordering algorithm on SpGEMM performance. We then replace the hash accumulator with the cache-aware hybrid accumulator for each approach and report the incremental gain as the stacked segment on each bar.

As shown in Fig. 10, with hash-based accumulation, LSH-based methods (including naive LSH and C-LSH) contribute to 80% of the fastest cases, with our C-LSH alone accounting for half of them. This result indicates that LSH-style techniques are effective for SpGEMM performance and that C-LSH preserves clustering quality while enforcing bounded candidate generation. Moreover, after introducing the cache-aware hybrid accumulation, all methods gain further improvements of up to 103%, 114%, 79%, 90%, 116%, 148%, 138%, respectively. The cases without noticeable acceleration typically lack enough compact clusters, thus falling back to pure hash accumulation. Overall, C-LSH achieves geometric-mean speedups of 2.37×, 1.49×, 1.42×, 1.48×, 1.14×, and 1.01× over the six competing algorithms, respectively. Compared with naive LSH, our C-LSH exhibits significant robustness, with a performance range of 0.85× to 1.29×, indicating an excellent trade-off between efficiency and clustering quality.

E. Preprocessing Overhead

To evaluate practicality, we measure the time and memory overhead of the two-phase clustering on the 20 representative matrices listed in Table II on the platform ③.

Time Overhead. Fig. 11 shows the amortization cost of different reordering methods, where the x-axis denotes the number of SpGEMM iterations required to amortize preprocessing, and the y-axis reports the fraction of matrices. As

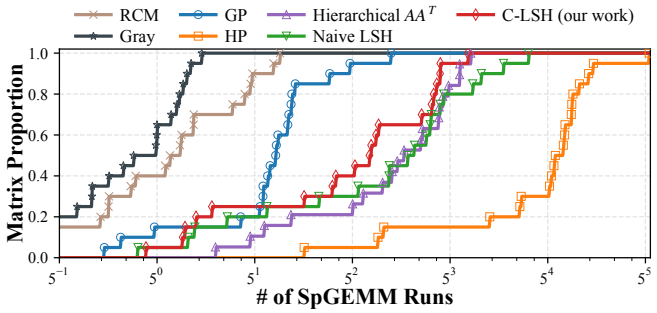


Fig. 11: The time overhead of reordering methods.

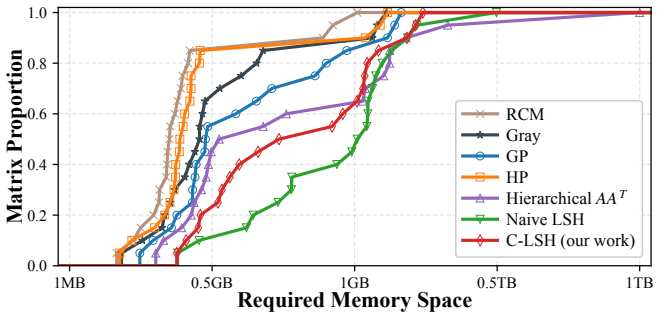


Fig. 12: The memory consumption of reordering methods.

can be seen, RCM and Gray introduce very low preprocessing overhead, with almost all matrices amortized within five iterations. GP amortizes 80% of the matrices within 10 iterations, whereas HP requires over 625 iterations for 70%. Conventional LSH and AA^T methods show similar trends, with nearly 50% of matrices amortized within 100 iterations. In contrast, C-LSH achieves lower time overhead due to its deterministic window constraint. In particular, for matrices with billion-scale intermediate results (from *case39* to *wikipedia-20070206* in Table II), C-LSH requires only 1.66 iterations on average.

Memory Consumption. Fig. 12 presents the peak memory requirements of the reordering methods. Typically, RCM, Gray, GP, and HP require less than 5 GB of memory for all matrices. In the worst case, LSH and AA^T encounter out-of-memory faults on *wikipedia*, exceeding 1 TB of memory demand. In contrast, C-LSH reduces the memory complexity from quadratic to near-linear, requiring only 20 GB of memory for this sparse matrix. For matrices with more than 1 billion floating-point operations, our C-LSH method requires only $0.61\times$ and $0.87\times$ memory on average compared with naive LSH and AA^T clustering, respectively. Overall, the bounded preprocessing overhead and stable memory consumption demonstrate the practicality of LeSpGEMM.

F. Runtime Breakdown of LeSpGEMM

Fig. 13 illustrates the runtime breakdown of LeSpGEMM over 20 representative matrices. As can be seen, the setup and memory allocation phase each accounts for less than 5% of the total runtime. The numeric phase (in blue) consistently

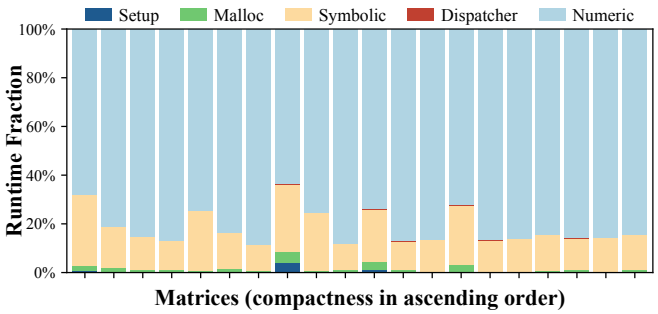


Fig. 13: The runtime breakdown of LeSpGEMM.

dominates execution, typically contributing 63%–88% of total cycles, and thus warrants careful data-locality optimization. Besides, the symbolic phase (in yellow) represents the primary analysis overhead, generally consuming 10% to 30% of runtime, which is a necessary trade-off for identifying the sparse structure of product C.

Crucially, our adaptive dispatcher (in red) introduces negligible computational overhead, consistently taking less than 0.5% of the total execution time across all matrices. This nearly zero cost is attributed to our hybrid accumulation design, which reuses metadata collected during the symbolic phase, enabling low-cost but effective decisions based on cache-resident working set size and cluster compactness.

V. Related Work

Optimizing SpGEMM has attracted growing attention in recent years. First, the inherent irregularity of sparse matrices makes **load balancing** a primary challenge in parallel SpGEMM. Early row-wise approaches primarily relied on row-binning strategies [29], [61], where rows with similar estimated products were grouped to mitigate workload divergence. Nagasaka et al. [30], [62] further extended this idea with multi-level blocking to improve cache reuse and vectorization. To achieve finer granularity, tiled partitioning methods such as TileSpGEMM [16], AmgT [3], and HSMU [34] partitioned matrices into uniform sparse blocks for better workload balance. Recently, several studies have explored adaptive scheduling optimizations. For instance, ApSpGEMM [20] introduced adaptive panel allocation to enhance parallelism on heterogeneous systems. Similarly, LSAF [63] proposed a dynamic packaging framework with runtime adjustments for multi-core systolic arrays. Compared with those methods, our LeSpGEMM introduces a cache-aware adaptive dispatcher that partitions tasks based on structural compactness and L2 cache residency. By explicitly considering the L2 cache budget, our approach achieves excellent data locality while avoiding cache thrashing and excessive memory traffic.

In addition, the performance of SpGEMM is largely dictated by **sparse accumulation strategies**. The Expansion, Sorting, and Compression (ESC) framework [21], [40], [41], [64] decouples the computation into localized sorting and merging primitives. Alternatively, heap-based accumulators [29], [30],

[65] and merge-based methods [66]–[68] are employed to process sorted input rows. With the rise of multi-core architectures, hash-based accumulation [31]–[36] has become the state-of-the-art on CPUs due to its average $O(1)$ complexity and superior flexibility. Moreover, the advent of wide SIMD units has facilitated the development of vectorized accumulators [33], [69]. However, traditional sparse accumulators often suffer from irregular memory access, while dense accumulators incur prohibitive zero-padding overhead for highly sparse patterns. In contrast, our LeSpGEMM addresses this issue by fully leveraging the structural density in compact clusters while remaining efficient in sparse regions.

Furthermore, **matrix reordering and clustering** are critical preprocessing steps to improve data locality [50], [60], [70]. Islam et al. [19] first validated the effectiveness of graph partitioning (GP) [23], [24] and hypergraph partitioning (HP) [26], and proposed a hierarchical clustering algorithm based on symbolic AA^T analysis. Subsequently, Yadav et al. [18] introduced a more sophisticated spectral clustering algorithm. In addition, locality-sensitive hashing (LSH) has been widely used for approximate nearest neighbor search and similarity detection [71]–[73], and has recently been applied to accelerate SpMM computation [27]. However, on large-scale skewed matrices, the naive LSH algorithm suffers from both high memory overhead and excessive computation due to the quadratic growth of candidate pairs in skewed buckets, leading to poor scalability. In contrast, our constrained-LSH method (C-LSH) addresses this challenge by implementing a localized sliding window search, achieving near-linear complexity and a robust memory footprint during preprocessing.

VI. Conclusion and Future Work

In this paper, we propose LeSpGEMM, a locality-enhanced SpGEMM algorithm that integrates lightweight clustering with cache-aware hybrid accumulation on CPUs. We first design C-LSH, which introduces coarse-grained locality for cluster-wise SpGEMM with near-linear complexity. Next, we develop a cache-aware hybrid accumulator to exploit the compactness within each cluster for modern vectorized architectures. The experimental results on three different platforms demonstrate that LeSpGEMM achieves significant speedups over six state-of-the-art implementations while providing an effective trade-off between preprocessing overhead and runtime acceleration compared to four existing reordering schemes. Future work includes adaptively selecting the optimal preprocessing strategies and extending our solutions to CPU-GPU heterogeneous collaborative computing.

Acknowledgment

We greatly appreciate the invaluable comments and suggestions from all the reviewers and shepherds. Chubo Liu is the corresponding author of this paper. This work was partially supported by the Creative Research Groups Program of the National Natural Science Foundation of China (Grant No. 62321003), the National Key R&D Program of China (Grant No. 2025YFB3003703), the National Natural Science Foundation of China (Grant Nos. 62502155, 62441234, 62532005, U25A20422, and 62541202), and the Natural Science Foundation of Hunan Province (Grant No. 2026JJ60510). This work was partly supported by the China Postdoctoral Science Foundation (Certificate Nos. 2026M791623 and 2026T190411).

References

[1] G. Ballard, C. Siefert, and J. Hu, "Reducing communication costs for sparse matrix multiplication within algebraic multigrid," *SIAM Journal on Scientific Computing*, vol. 38, no. 3, pp. C203–C231, 2016. [Online]. Available: https://doi.org/10.1137/15M1028807

[2] R. Li, B. Sjogreen, and U. M. Yang, "A new class of amg interpolation methods based on matrix-matrix multiplications," *SIAM Journal on Scientific Computing*, vol. 43, no. 5, pp. S540–S564, 2021. [Online]. Available: https://doi.org/10.1137/20M134931X

[3] Y. Lu, L. Zeng, T. Wang, X. Fu, W. Li, H. Cheng, D. Yang, Z. Jin, M. Casas, and W. Liu, "AmgT: Algebraic multigrid solver on tensor cores," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1–16. [Online]. Available: https://doi.org/10.1109/SC41406.2024.00058

[4] A. Azad, G. A. Pavlopoulos, C. A. Ouzounis, N. C. Kyrpides, and A. Buluç, "HipMCL: a high-performance parallel implementation of the markov clustering algorithm for large-scale networks," *Nucleic Acids Research*, vol. 46, no. 6, pp. e33–e33, 04 2018. [Online]. Available: https://doi.org/10.1093/nar/gkx1313

[5] T. A. Davis, "Algorithm 1000: SuiteSparse:GraphBLAS: Graph algorithms in the language of sparse linear algebra," *ACM Trans. Math. Softw.*, vol. 45, no. 4, Dec. 2019. [Online]. Available: https://doi.org/10.1145/3322125

[6] J. Li, F. Wang, T. Araki, and J. Qiu, "Generalized sparse matrix-matrix multiplication for vector engines and graph applications," in *2019 IEEE/ACM Workshop on Memory Centric High Performance Computing (MCHPC)*. IEEE, 2019, pp. 33–42. [Online]. Available: https://doi.org/10.1109/MCHPC49590.2019.00012

[7] T. A. Davis, "Algorithm 1037: SuiteSparse:GraphBLAS: Parallel graph algorithms in the language of sparse linear algebra," *ACM Trans. Math. Softw.*, vol. 49, no. 3, Sep. 2023. [Online]. Available: https://doi.org/10.1145/3577195

[8] H. Lee, M. Jain, and S. Ghosh, "Sparse deep neural network inference using different programming models," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2022, pp. 1–6. [Online]. Available: https://doi.org/10.1109/HPEC55821.2022.9926362

[9] I. Tasou, P. Mpakos, A. Vlachos, D. Adamopoulos, G. Giannakopoulos, K. Katsikopoulos, I. Karaparisis, M. Lazou, S. Loukovitis, A. Mei *et al.*, "Sparse computations in deep learning inference," 2025. [Online]. Available: https://arxiv.org/abs/2512.02550

[10] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "SIGMA: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 58–70. [Online]. Available: https://doi.org/10.1109/HPCA47549.2020.00015

[11] O. Selvitopi, M. T. Hussain, A. Azad, and A. Buluç, "Optimizing high performance markov clustering for pre-exascale architectures," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2020, pp. 116–126. [Online]. Available: https://doi.org/10.1109/IPDPS47924.2020.00022

[12] C. Zhang, M. Bremer, C. Chan, J. Shalf, and X. Guo, "ASA: Accelerating sparse accumulation in column-wise spgemm," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 4, Sep. 2022. [Online]. Available: https://doi.org/10.1145/3543068

[13] J. Gao, W. Ji, F. Chang, S. Han, B. Wei, Z. Liu, and Y. Wang, "A systematic survey of general sparse matrix-matrix multiplication," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–36, 2023. [Online]. Available: https://doi.org/10.1145/3571157

[14] E. Bank Tavakoli, M. Riera, M. H. Quraishi, and F. Ren, "FSpGEMM: A framework for accelerating sparse general matrix–matrix multiplication using gustavson's algorithm on fpgas," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 4, pp. 633–644, 2024. [Online]. Available: https://doi.org/10.1109/TVLSI.2024.3355499

[15] G. Zhang, N. Attaluri, J. S. Emer, and D. Sanchez, "Gamma: Leveraging gustavson's algorithm to accelerate sparse matrix multiplication," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 687–701. [Online]. Available: https://doi.org/10.1145/3445814.3446702

[16] Y. Niu, Z. Lu, H. Ji, S. Song, Z. Jin, and W. Liu, "TileSpGEMM: a tiled algorithm for parallel sparse general matrix-matrix multiplication on gpus," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 90–106. [Online]. Available: https://doi.org/10.1145/3503221.3508431

[17] Y. Hong and A. Buluç, "A sparsity-aware distributed-memory algorithm for sparse-sparse matrix multiplication," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*. IEEE, 2024, pp. 1–14. [Online]. Available: https://doi.org/10.1109/SC41406.2024.00053

[18] S. Yadav and B. Asgari, "Bootes: Boosting the efficiency of sparse accelerators using spectral clustering," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 809–823. [Online]. Available: https://doi.org/10.1145/3725843.3756125

[19] A. A. R. Islam, H. Xu, D. Dai, and A. Buluç, "Improving spgemm performance through matrix-reordering and cluster-wise computation," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1446–1463. [Online]. Available: https://doi.org/10.1145/3712285.3759885

[20] D. Yao, S. Zhao, T. Liu, G. Wu, and H. Jin, "ApSpGEMM: Accelerating large-scale spgemm with heterogeneous collaboration and adaptive panel," *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 1, pp. 1–23, 2025. [Online]. Available: https://doi.org/10.1145/3703352

[21] J. Wolfson-Pou, J. Laukemann, and F. Petrini, "MAGNUS: Generating data locality to accelerate sparse matrix-matrix multiplication on cpus," in *Proceedings of the 39th ACM International Conference on Supercomputing*, 2025, pp. 442–457. [Online]. Available: https://doi.org/10.1145/3721145.3725773

[22] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software (TOMS)*, vol. 4, no. 3, pp. 250–269, 1978. [Online]. Available: https://doi.org/10.1145/355791.355796

[23] G. Karypis and V. Kumar, "METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices," 1997. [Online]. Available: https://hdl.handle.net/11299/215346

[24] G. Karypis, K. Schloegel, and V. Kumar, "ParMETIS," *Parallel graph partitioning and sparse matrix ordering library. Version*, vol. 2, 2003. [Online]. Available: http://charm.cs.uiuc.edu/users/gupta/2012_CloudHPCLB_charm/src/libs/ck-libs/parmetis/Manual/manual.pdf

[25] G. Echbarthi and H. Kheddouci, "Streaming METIS partitioning," in *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 2016, pp. 17–24. [Online]. Available: https://doi.org/10.1109/ASONAM.2016.7752208

[26] G. Ballard, A. Druinsky, N. Knight, and O. Schwartz, "Hypergraph partitioning for sparse matrix-matrix multiplication," *ACM Transactions on Parallel Computing (TOPC)*, vol. 3, no. 3, pp. 1–34, 2016. [Online]. Available: https://doi.org/10.1145/3015144

[27] P. Jiang, C. Hong, and G. Agrawal, "A novel data transformation and execution strategy for accelerating sparse matrix multiplication on gpus," in *Proceedings of the 25th ACM SIGPLAN symposium on principles and practice of parallel programming*, 2020, pp. 376–388. [Online]. Available: https://doi.org/10.1145/3332466.3374546

[28] U. V. Catalyurek and C. Aykanat, "Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication," *IEEE Transactions on parallel and distributed systems*, vol. 10, no. 7, pp. 673–693, 2002. [Online]. Available: https://doi.org/10.1109/71.780863

[29] W. Liu and B. Vinter, "An efficient gpu general sparse matrix-matrix multiplication for irregular data," in *2014 IEEE 28th international parallel and distributed processing symposium*. IEEE, 2014, pp. 370–381. [Online]. Available: https://doi.org/10.1109/IPDPS.2014.47

[30] Y. Nagasaka, S. Matsuoka, A. Azad, and A. Buluç, "Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors," *Parallel Computing*, vol. 90, p. 102545, 2019. [Online]. Available: https://doi.org/10.1016/j.parco.2019.102545

[31] M. Deveci, C. Trott, and S. Rajamanickam, "Performance-portable sparse matrix-matrix multiplication for many-core architectures," in *2017 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2017, pp. 693–702. [Online]. Available: https://10.1109/IPDPSW.2017.8

[32] H. Cheng, W. Li, Y. Lu, and W. Liu, "HASpGEMM: Heterogeneity-aware sparse general matrix-matrix multiplication on modern asymmetric multicore processors," in *Proceedings of the 52nd International Conference on Parallel Processing*, 2023, pp. 807–817. [Online]. Available: https://doi.org/10.1145/3605573.3605611

[33] V. Le Fèvre and M. Casas, "Efficient execution of spgemm on long vector architectures," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, 2023, pp. 101–113. [Online]. Available: https://doi.org/10.1145/3588195.3593000

[34] M. Wu, H. Luo, F. Li, Y. Zhang, Z. Tang, K. Li, J. Zhang, and C. Liu, "HSMU-SpGEMM: Achieving high shared memory utilization for parallel sparse general matrix-matrix multiplication on modern gpus," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1452–1466. [Online]. Available: https://doi.org/10.1109/HPCA61900.2025.00109

[35] S. Yang, Y. Wang, L. Li, Q. Peng, and Q. Yu, "MH-SpGEMM: Efficient sparse general matrix-matrix multiplication on modern gpus via masking and hashing cooperative optimization," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 2025, pp. 807–814. [Online]. Available: https://doi.org/10.1109/ICCD65941.2025.00119

[36] Y. Li, Z. Pan, P. Qu, and Y. Zhang, "VDHA: Vector-driven hash aggregation for sparse matrix-sparse vector multiplication on gpus," in *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2026, pp. 259–272. [Online]. Available: https://doi.org/10.1145/3774934.3786447

[37] M. Parger, M. Winter, D. Mlakar, and M. Steinberger, "SpECK: Accelerating gpu sparse matrix-matrix multiplication through lightweight analysis," in *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2020, pp. 362–375. [Online]. Available: https://doi.org/10.1145/3332466.3374521

[38] G. Zhao and Y. Cao, "Highly efficient load-balanced dataflow for spgemms on systolic arrays," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, 2024, pp. 272–276. [Online]. Available: https://doi.org/10.1145/3649476.3658777

[39] Q. Wang, Y. Wang, Y. Luo, R. Luo, and P. Tang, "HR-SpMM: Adaptive row partitioning and hybrid kernel design for sparse matrix multiplication," in *Proceedings of the 39th ACM International Conference on Supercomputing*, 2025, pp. 161–172. [Online]. Available: https://doi.org/10.1145/3721145.3725770

[40] J. Liu, X. He, W. Liu, and G. Tan, "Register-aware optimizations for parallel sparse matrix–matrix multiplication," *International Journal of Parallel Programming*, vol. 47, no. 3, pp. 403–417, 2019. [Online]. Available: https://doi.org/10.1007/s10766-018-0604-8

[41] E. Lorimer, R. Han, S. H. Kang, and H. Kim, "Multiway merge partitioning for sparse-sparse matrix multiplication on gpus," in *2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2025, pp. 160–171. [Online]. Available: https://doi.org/10.1109/PACT65351.2025.00025

[42] T. A. Davis and Y. Hu, "The university of florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, Dec. 2011. [Online]. Available: https://doi.org/10.1145/2049662.2049663

[43] Intel Corporation, *Intel® oneAPI Math Kernel Library*, Intel Corporation, 2025, version 2025.3.1. [Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html

[44] AMD, "AOCL-Sparse: Basic linear algebra subroutines for sparse matrices and vectors," AMD Developer Central, 2025, version 5.2, Accessed: 2025-12-31. [Online]. Available: https://www.amd.com/en/developer/aocl/sparse.html

[45] M. Deveci, C. Trott, and S. Rajamanickam, "Multithreaded sparse matrix-matrix multiplication for many-core and gpu architectures," *Parallel Computing*, vol. 78, pp. 33–46, 2018. [Online]. Available: https://doi.org/10.1016/j.parco.2018.06.009

[46] S. Rajamanickam, S. Acer, L. Berger-Vergiat, V. Dang, N. Ellingwood, E. Harvey, B. Kelley, C. R. Trott, J. Wilke, and I. Yamazaki, "Kokkos Kernels: Performance portable sparse/dense linear algebra and graph kernels," *arXiv preprint arXiv:2103.11991*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2103.11991

[47] X. An and Ü. V. Çatalyürek, "Column-segmented sparse matrix-matrix multiplication on multicore cpus," in *2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, 2021, pp. 202–211. [Online]. Available: https://doi.org/10.1109/HiPC53243.2021.00034

[48] P. Jaccard, "The distribution of the flora in the alpine zone," *The New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912. [Online]. Available: https://doi.org/10.1111/j.1469-8137.1912.tb05611.x

[49] A. Pinar and M. T. Heath, "Improving performance of sparse matrix-vector multiplication," in *Proceedings of the 1999 ACM/IEEE conference on Supercomputing*, 1999, pp. 30–es. [Online]. Available: https://doi.org/10.1145/331532.331562

[50] J. D. Trotter, S. Ekmekçibaşı, J. Langguth, T. Torun, E. Düzakın, A. Ilic, and D. Unat, "Bringing Order to Sparsity: A sparse matrix reordering study on multicore cpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–13. [Online]. Available: https://doi.org/10.1145/3581784.3607046

[51] A. Clauset, C. R. Shalizi, and M. E. Newman, "Power-law distributions in empirical data," *SIAM review*, vol. 51, no. 4, pp. 661–703, 2009. [Online]. Available: https://doi.org/10.1137/070710111

[52] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of massive data sets*. Cambridge university press, 2020. [Online]. Available: https://doi.org/10.1017/cbo9781139924801

[53] M. Singh, "Scalability and sparsity issues in recommender datasets: a survey," *Knowledge and Information Systems*, vol. 62, no. 1, pp. 1–43, 2020. [Online]. Available: https://doi.org/10.1007/s10115-018-1254-2

[54] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse gpu kernels for deep learning," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–14. [Online]. Available: https://doi.org/10.1109/SC41405.2020.00021

[55] S. Lin, C. Liu, Y. Ding, J. T. Zhou, K. Li, and W. Yang, "SSpMV: A sparsity-aware spmv framework empowered by multimodal machine learning," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–7. [Online]. Available: https://doi.org/10.1109/DAC63849.2025.11132896

[56] W. Wu, B. Li, L. Chen, J. Gao, and C. Zhang, "A review for weighted minhash algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 6, pp. 2553–2573, 2020. [Online]. Available: https://doi.org/10.1109/TKDE.2020.3021067

[57] A. C. De Melo, "The new linux'perf'tools," in *Slides from Linux Kongress*, vol. 18, no. 1, 2010. [Online]. Available: http://oldvger.kernel.org/~acme/perf/lk2010-perf-acme.pdf

[58] J. Treibig, G. Hager, and G. Wellein, "LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments," in *2010 39th international conference on parallel processing workshops*. IEEE, 2010, pp. 207–216. [Online]. Available: https://doi.org/10.1109/ICPPW.2010.38

[59] T. Roehl, J. Treibig, G. Hager, and G. Wellein, "Overhead analysis of performance counter measurements," in *2014 43rd international conference on parallel processing workshops*. IEEE, 2014, pp. 176–185. [Online]. Available: https://doi.org/10.1109/ICPPW.2014.34

[60] W.-H. Liu and A. H. Sherman, "Comparative analysis of the cuthill–mckee and the reverse cuthill–mckee ordering algorithms for sparse matrices," vol. 13, no. 2, p. 198–213, Apr. 1976. [Online]. Available: https://doi.org/10.1137/0713020

[61] W. Liu and B. Vinter, "A framework for general sparse matrix–matrix multiplication on gpus and heterogeneous processors," *Journal of Parallel and Distributed Computing*, vol. 85, pp. 47–61, 2015. [Online]. Available: https://doi.org/10.1016/j.jpdc.2015.06.010

[62] Y. Nagasaka, S. Matsuoka, A. Azad, and A. Buluç, "High-performance sparse matrix-matrix products on Intel KNL and multicore architectures," in *Workshop Proceedings of the 47th International Conference on Parallel Processing*, 2018, pp. 1–10. [Online]. Available: https://doi.org/10.1145/3229710.3229720

[63] Y. Cao, H. Jiang, G. Zhao, D. Shi, R. Zhang, and W. Wang, "LSAF: A load-balancing spgemm acceleration framework with dynamic package and static partition for multi-core systolic arrays," *Parallel Computing*, vol. 127, p. 103186, 2026. [Online]. Available: https://doi.org/10.1016/j.parco.2026.103186

[64] R. Kunchum, A. Chaudhry, A. Sukumaran-Rajam, Q. Niu, I. Nisa, and P. Sadayappan, "On improving performance of sparse matrix-matrix multiplication on gpus," in *Proceedings of the International Conference on Supercomputing*, ser. ICS '17. New York, NY, USA: Association for Computing Machinery, 2017. [Online]. Available: https://doi.org/10.1145/3079079.3079106

[65] A. Azad, G. Ballard, A. Buluc, J. Demmel, L. Grigori, O. Schwartz, S. Toledo, and S. Williams, "Exploiting multiple levels of parallelism in sparse matrix-matrix multiplication," *SIAM Journal on Scientific Computing*, vol. 38, no. 6, pp. C624–C651, 2016. [Online]. Available: https://doi.org/10.1137/15M104253X

[66] S. Dalton, S. Baxter, D. Merrill, L. Olson, and M. Garland, "Optimizing sparse matrix operations on gpus using merge path," in *2015 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 2015, pp. 407–416. [Online]. Available: https://doi.org/10.1109/IPDPS.2015.98

[67] F. Gremse, K. Kupper, and U. Naumann, "Memory-efficient sparse matrix-matrix multiplication by row merging on many-core architectures," *SIAM Journal on Scientific Computing*, vol. 40, no. 4, pp. C429–C449, 2018. [Online]. Available: https://doi.org/10.1137/17M1121378

[68] Z. Gu, J. Moreira, D. Edelsohn, and A. Azad, "Bandwidth optimized parallel algorithms for sparse matrix-matrix multiplication using propagation blocking," in *Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures*, 2020, pp. 293–303. [Online]. Available: https://doi.org/10.1145/3350755.3400216

[69] C. Hong, Q. Wang, X. Peng, G. Liu, Q. Zhang, X. Chen, and J. Liu, "VES: Vectorized sparse general matrix-matrix multiplication on multi-core dsps," in *Proceedings of the 54th International Conference on Parallel Processing*, ser. ICPP '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 135–145. [Online]. Available: https://doi.org/10.1145/3754598.3754613

[70] B. O. F. Auer and R. H. Bisseling, "Efficient matching for column intersection graphs," *ACM J. Exp. Algorithmics*, vol. 19, Jan. 2015. [Online]. Available: https://doi.org/10.1145/2616587

[71] D. D. Le and H. W. Lauw, "Stochastically robust personalized ranking for lsh recommendation retrieval," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 4594–4601. [Online]. Available: https://doi.org/10.1609/aaai.v34i04.5889

[72] J. Wu, L. Shen, and L. Liu, "LSH-based distributed similarity indexing with load balancing in high-dimensional space: J. wu et al." *The Journal of Supercomputing*, vol. 76, no. 1, pp. 636–665, 2020. [Online]. Available: https://doi.org/10.1007/s11227-019-03047-6

[73] W. Lyu, R. Sridharamurthy, J. M. Phillips, and B. Wang, "Fast comparative analysis of merge trees using locality sensitive hashing," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 1, pp. 141–151, 2024. [Online]. Available: https://doi.org/10.1109/TVCG.2024.3456383